

# Inter-Service Communication

How microservices find, talk to, and work with each other

Sync vs Async

HTTP/REST

Message Queues

Service Discovery

Data Isolation

# What We'll Cover

01

## Communication Overview

Synchronous vs asynchronous — the two ways services talk

03

## Asynchronous Communication

Message queues — fire and forget, no waiting

05

## Data Isolation

Why each service must own its own database

02

## Synchronous Communication

HTTP/REST — services calling each other directly

04

## Service Discovery

How services find each other at runtime  
(preview)

# Two Ways Services Talk

## ⚡ Synchronous

HTTP / REST

Service A calls Service B and waits for the response before continuing.



- ✓ Simple to implement
- ✓ Easy to debug & trace
- ✗ Caller is blocked while waiting
- ✗ Tight runtime dependency

## ✉ Asynchronous

Message Queue (RabbitMQ, Azure Service Bus...)

Service A publishes an event and moves on. Service B picks it up whenever it's ready.



- ✓ Caller doesn't wait (non-blocking)
- ✓ Services are fully decoupled
- ✗ Harder to debug & trace
- ✗ More infrastructure needed

# HTTP Calls Between Services

From our C# store example — OrderService calling ProductService to check stock:

OrderService.cs

```
// Step 1 – register the HTTP client
builder.Services.AddHttpClient("ProductService",
    c => c.BaseAddress = new Uri("http://localhost:5001"));

// Step 2 – call ProductService
var client = httpFactory.CreateClient("ProductService");
var product = await client.GetFromJsonAsync<Product>(
    $"/products/{req.ProductId}");
```

1

## Register the client

Tell ASP.NET where ProductService lives

2

## Inject the factory

IHttpClientFactory creates a configured client

3

## Make the HTTP call

GET /products/{id} → deserialize JSON

4

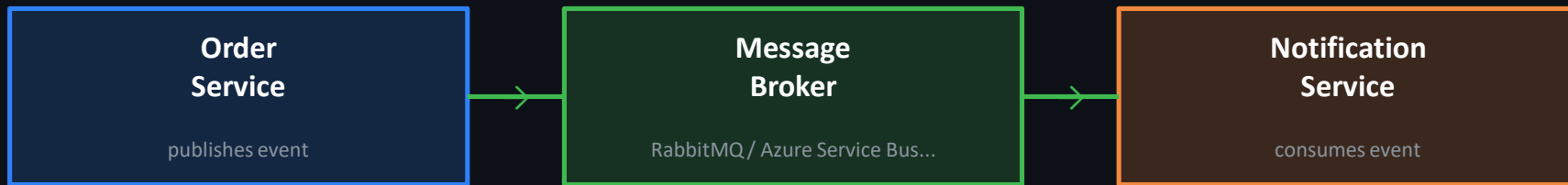
## Handle the result

If null → product doesn't exist → return 400

⚠ What if ProductService is down? → OrderService fails too. This is the risk of synchronous calls.

# Events & Message Queues

*Scenario: order is placed → notification should be sent. Does OrderService really need to wait for the email?*



## Event: "OrderPlaced"

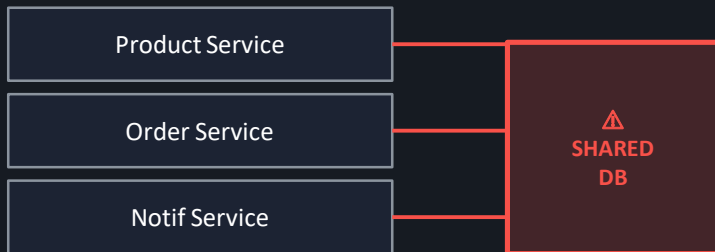
```
{ "orderId": 42,  
  "customerEmail": "student@example.com",  
  "productName": "Laptop",  
  "total": 2599.98 }
```

- ✓ **OrderService finishes instantly**  
doesn't wait
- ✓ **NotificationService can be down**  
event is queued and processed later
- ✓ **Add more consumers (SMS, push)**  
without changing OrderService

# Each Service Owns Its Data

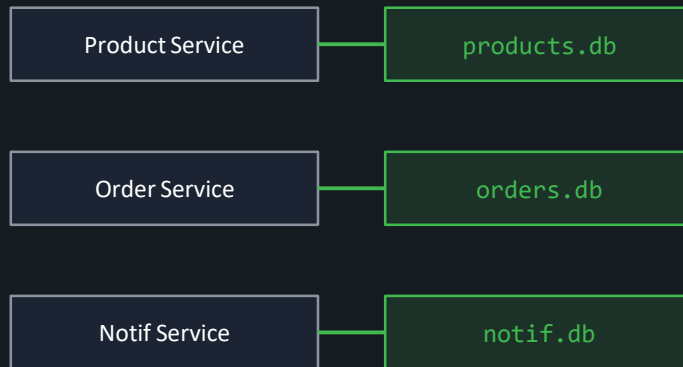
*Rule: no two services may share a database or call each other's database directly.*

## ✗ Wrong — Shared DB



- ✗ Schema change breaks all services
- ✗ One slow query affects everyone
- ✗ Can't scale services independently

## ✓ Correct — DB per Service



- ✓ Each service can use a different DB type
- ✓ Schema changes stay isolated
- ✓ Scale storage independently per service

# Why Split Your Database?

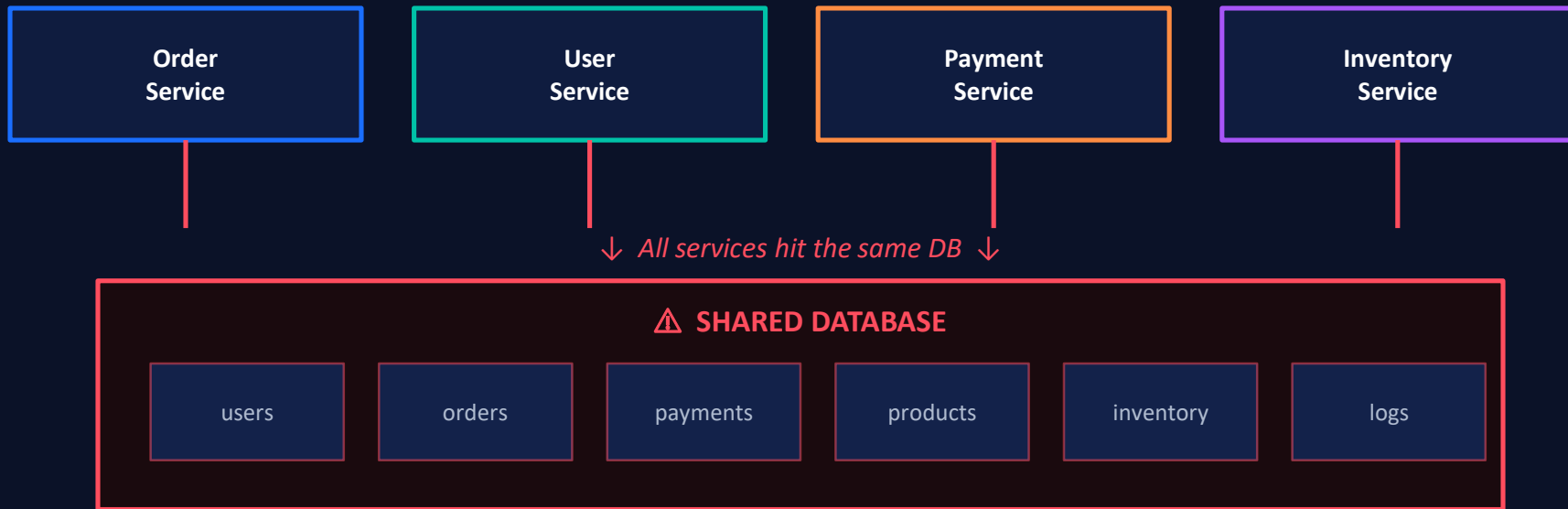
*From one shared database to database-per-service*

Schema Coupling

Scaling Problems

Data Ownership

# The Shared Database Trap



*Nobody owns anything. Everybody touches everything. One change can break the whole system.*



# Schema Change = Everyone's Problem

```
-- You rename a column in Orders
```

```
ALTER TABLE Orders  
RENAME COLUMN user_id TO customer_id
```



X Payment Service breaks

X Shipping Service breaks

X Reports Service breaks

X Analytics breaks

## Coordination Tax

Every DB change needs sign-off from ALL teams before deployment

## Fear of Change

Developers avoid refactoring because something unknown might break

## Deployment Blocked

Team A can't ship until Team B finishes their pending migration

*"You're afraid to delete ANYTHING because something, somewhere might depend on it."*

## PAIN POINT # 2

# One Slow Query Kills Everyone

ProductService  
runs heavy report

Locks Products  
table 30 seconds

OrderService  
tries to read Products

OrderService  
hangs completely

 Users can't place orders — everything is down

## PAIN POINT # 3

# You Can't Scale Independently

 **Black Friday**

Orders traffic  $\times 100$  — but you must scale the ENTIRE shared DB including Users, Products, Payments

 **Very Expensive**

Scaling a single massive DB is costly — you pay for resources used by services that don't need it

 **Tech Lock-in**

All services forced to use same DB tech — can't pick MongoDB for Users or Elasticsearch for Products

# Database per Service

**Golden Rule: Each service owns its data exclusively — no other service can touch it directly.**



✓ No shared tables

✓ No cross-service joins

✓ Communicate only through APIs

# Shared DB vs Database per Service

|                | ✗ Shared Database   | ✓ DB per Service         |
|----------------|---------------------|--------------------------|
| Schema change  | Breaks all services | Only affects one service |
| Slow query     | All services hang   | Isolated to one service  |
| Deployment     | Everyone waits      | Teams ship independently |
| Scaling        | Scale entire DB     | Scale only what's needed |
| Data ownership | Nobody / everybody  | Clear per service        |
| Tech choice    | One DB for all      | Right tool per service   |

## *"How do I get data from multiple services?"*

### Option A — API Composition

Client asks for order + user details

- 1 API Gateway receives request
- 2 Calls OrderService → gets order data
- 3 Calls UserService → gets user data
- 4 Merges & returns to client

✓ Simple • No messaging needed • Good for reads

### Option B — Events (Messaging)

User updates email → all services sync

- 1 UserService updates email in its DB
- 2 Publishes event: UserEmailUpdated
- 3 RabbitMQ / Service Bus delivers event
- 4 OrderService listens & updates local copy

✓ Decoupled • Eventual consistency • RabbitMQ in .NET

# Key Takeaways

01

## Sync vs Async

Two ways services talk: HTTP (waits for response) and message queues (non-blocking, fully decoupled).

02

## Synchronous (HTTP)

Services call each other directly and wait. Simple to debug but creates runtime dependencies.

03

## Asynchronous (Events)

Services publish events to a broker. Consumers react at their own pace — fully decoupled.

04

## Data Isolation

One database per service. No sharing. Ever. This is what makes true isolation possible.